DEFENSIVE PUBLICATION — DP-001

Title: Tamper-Evident, Non-Downgradeable Binding of Advisory Header Metadata to Ciphertext in Client-Side-Encrypted Document Envelopes via AES-GCM Additional Authenticated Data

Author: David Brown
Assignee / Defensive Publisher: Enthropic Data LLC, Weddington, North Carolina, USA
Intended venue: Technical Disclosure Commons (TDCommons.org)
Submission date: 2026-08-15

Priority-date anchors (independent of this submission):

`docs/SPEC.md` — SHA-256:

`2e1ac039bfb1920bc5ba848f2bfdf904e0c9ccfe0443c016034a6eb5bc7c6c41`

Anchored by OpenTimestamps (`docs/defensive-pub/anchors/SPEC.md.ots`), stamped 2026-07-18, Bitcoin-confirmed in blocks 958650 and 958690.

`cell-crypto.js` (reference implementation) — SHA-256:

`11ce4b07f2459ea1b4677d98e864a424849afd7080cafdfdff60ca7b81decae5`

Anchored by OpenTimestamps (`docs/defensive-pub/anchors/cell-crypto.js.ots`), stamped 2026-07-18, Bitcoin-confirmed in blocks 958650 and 958690.

The anchored bytes themselves sit beside their proofs in `docs/defensive-pub/anchors/`, so both digests above can be reproduced without a repository checkout. The files at the paths `docs/SPEC.md` and `cell-crypto.js` have since advanced to v1.3 and hash differently; they carry their own, later anchor.

Public repository, published 2026-08-15:

`https://github.com/Enthropic-Data-LLC/cd-vanilla-public`

The anchored bytes and their proofs are in `docs/defensive-pub/anchors/`; the originating tag `cell-format-v1.2-defensive-pub` (commit `62a8143`) is in the private source repository and is not needed — the proofs commit to file content, not to git history. Verify any anchor with: `ots verify docs/defensive-pub/anchors/SPEC.md.ots`

Abstract

An encrypted-document envelope carries advisory metadata — a threshold/quorum requirement, a lifetime policy, a distribution policy, and a lineage pointer — alongside ciphertext. Conventionally this metadata sits in a plaintext or merely hashed header, where a third party can strip it, alter it, or downgrade the format to disable the behavior it

governs, because the metadata is not bound to the ciphertext itself. An unkeyed header hash does not solve this: an attacker who edits the header simply recomputes the hash.

This disclosure describes binding that metadata to the ciphertext as AES-GCM **Additional Authenticated Data (AAD)**, computed over a deterministic canonical serialization, such that any modification, removal, or format-version downgrade of the bound metadata causes authenticated decryption to fail. The binding is **not strippable** — the recipient always reconstructs the AAD from the current header, so there is no separate field an attacker can delete to disable the check — and **not downgradeable** — decrypting without the AAD fails the authentication tag, and re-encrypting is impossible without the content encryption key. The mechanism is version-gated so that envelopes written under earlier rules remain verifiable unchanged.

The disclosed technique applies to any client-side-encrypted or end-to-end-encrypted document format that carries governance metadata alongside ciphertext, and is independent of how the content key is distributed to recipients.

Field

End-to-end and client-side encryption of documents; authenticated encryption with associated data (AEAD); tamper-evident metadata; zero-knowledge document storage.

Background and Problem Addressed

Client-side-encrypted document formats commonly attach governance metadata to ciphertext in a header: an expiry or release time, a quorum threshold specifying how many parties must cooperate to decrypt, redistribution or watermarking policy, and a pointer linking the document to a predecessor version.

Three deficiencies arise in the conventional arrangement:

1. **Plaintext metadata is freely editable.** A storage provider or network intermediary can alter an expiry date or reduce a quorum threshold with no detectable trace.
2. **An unkeyed hash over the header does not confer authenticity.** It detects accidental corruption, but an attacker who intentionally edits the header recomputes the hash trivially. Authenticity requires either a signature or a keyed binding.
3. **A signature alone remains downgrade-exposed.** Even where a header signature is present, an attacker may alter a format-version field so that a more permissive legacy code path ignores or fails to check the metadata. Where the signature is optional (as it commonly is, to support anonymous senders), unsigned envelopes retain deficiency (1) entirely.

The problem is to make advisory metadata tamper-evident, non-strippable, and non-downgradeable, **without** introducing a separate keyed MAC, a mandatory signature, or

a server-side enforcement point — the last being unavailable by construction in a zero-knowledge system where the server cannot read the content.

Summary of the Disclosed Mechanism

Fix the governing metadata **before** content encryption, serialize it canonically, and supply it as the AAD input to the AEAD used to encrypt the document body. The AES-GCM authentication tag then covers both the ciphertext and the metadata. At decrypt time the recipient reconstructs the AAD from the *current* header and supplies it; any mismatch — including an attempted downgrade to a no-AAD code path — fails tag verification and no plaintext is released.

The metadata remains readable in the header (it must be, to be acted upon before decryption), but it is no longer alterable.

Detailed Description

Primitives and parameters

- **Content cipher:** AES-256-GCM. 256-bit content encryption key (CEK) from a cryptographic RNG; 96-bit random initialization vector; 128-bit authentication tag.
- **Canonical serialization:** JSON with object keys sorted lexicographically at every level of nesting, no insignificant whitespace, primitive encoding and string escaping identical to standard JSON serialization, keys with undefined values omitted, and array order preserved with undefined elements encoded as null.

Canonicalization is specified as:

```
canonicalize(v):

  if v is null or not an object:
      return json_encode(v)

  if v is an array:
      return "["
           + join(",", [ canonicalize(e if e is defined else null)
                         for e in v ])
           + "]"

  otherwise (object):
      // keys sorted lexicographically; undefined values omitted
      keys = sort([ k for k in keys(v) if v[k] is defined ])
      return "{"
           + join(",", [ json_encode(k) + ":" + canonicalize(v[k])
                         for k in keys ])
           + "}"
```

Canonicalization ensures that re-serializing the envelope — pretty-printing it, or round-tripping it through a JSON library that reorders keys — does not alter the authenticated bytes and therefore does not produce a spurious authentication failure on an otherwise-untampered document.

Data structures

The bound metadata is assembled as a fixed-order array:

```
meta = [ prev_hash, threshold, lifetime, policy ]
```

where:

- `prev_hash` — lineage pointer to a predecessor envelope (hash of the predecessor's header), or null for an original document;
- `threshold` — `{ required: M, of_total: N }`, the quorum requirement;
- `lifetime` — `{ type, expires_at, release_at, on_expiry, single_use, minimum_atl }`, the temporal policy;
- `policy` — `{ copy_protection, watermark_mode, created_on_origin, origin_sig }`, the distribution/presentation policy.

Construction (encrypt path)

1. Generate the CEK (256-bit, cryptographic RNG) and IV (96-bit, cryptographic RNG).
2. Assemble `meta` from the finalized header fields. These fields are known prior to encryption.
3. Compute `AAD = UTF-8 bytes of canonicalize(meta)`.
4. Encrypt the document body: `ciphertext ‖ tag = AES-256-GCM(key = CEK, iv = IV, aad = AAD, plaintext = body)`
5. Store the IV and ciphertext in the envelope payload; store the `meta` field values in the header in readable form.

The body supplied at step 4 may itself be a composite structure; in the reference implementation it is a length-prefixed metadata manifest concatenated with the file bytes and then compressed, so that filename, MIME type, and size are also confidential — but that composition is incidental to the mechanism disclosed here.

Verification (decrypt path)

1. Reconstruct `AAD = UTF-8 bytes of canonicalize([prev_hash, threshold, lifetime, policy])` from the **current** header contents.
2. Recover the CEK by the envelope's key-distribution mechanism (out of scope here).
3. Decrypt: `body = AES-256-GCM-open(CEK, IV, AAD, ciphertext ‖ tag)`.
4. If any bound field was altered or removed, or if the envelope was downgraded such that a different AAD (or an empty AAD) is supplied, tag verification fails and no plaintext is released.

Fields deliberately excluded from the AAD

Fields that do not exist at encryption time cannot be bound as AAD. In the reference arrangement these are (a) the per-recipient access map, which wraps the freshly generated CEK and therefore is derived after the CEK exists, and (b) the payload hash, which is derived from the resulting ciphertext. These are instead covered by a hash and optional signature computed over the complete header after assembly. This split — AAD for pre-encryption metadata, header hash/signature for post-encryption metadata — is part of the disclosed arrangement.

Version gating

The serialization and AAD rules are selected by the envelope's own declared format version, so envelopes written under earlier rules continue to verify under those rules. In the reference implementation: version 1.0 envelopes carry no AAD and use non-canonical serialization; version 1.1 envelopes bind AAD over a non-canonical serialization; version 1.2 envelopes bind AAD over the canonical serialization above and use it for the header hash and signature as well. An envelope declaring a version outside the supported set is rejected outright rather than processed under a default rule — this rejection is itself a necessary part of the anti-downgrade property.

Security properties obtained

- **Tamper-evidence without a signature.** Metadata integrity does not depend on the optional sender signature, so it holds for anonymous/unsigned envelopes.
- **Non-strippable.** The verifier derives the AAD from whatever metadata the header currently carries. There is no flag, field, or toggle whose removal disables the check; removing the metadata changes the derived AAD and fails the tag.
- **Non-downgradeable.** A ciphertext produced with AAD cannot be opened without that same AAD. Editing the version field to select a no-AAD path fails the tag, and producing a genuinely no-AAD ciphertext requires the CEK, which the attacker does not hold.
- **No server-side enforcement point required.** The property holds in a zero-knowledge deployment where the storage provider cannot read content.

Acknowledged limitation

The mechanism binds metadata against modification by third parties — storage providers, network intermediaries, and any party without the CEK. It does **not**, and cannot, constrain a legitimate recipient who has recovered the CEK: such a party can necessarily decrypt regardless of what an expiry or policy field states. Lifetime and policy therefore remain advisory toward authorized holders. This limitation is inherent to any client-side-enforced policy scheme and is disclosed here explicitly to avoid overstating the property.

Variations and Alternatives

The following variants are disclosed as part of this publication and are intended to be covered as prior art:

- **Alternative AEADs.** Any AEAD accepting associated data may substitute for AES-256-GCM, including AES-GCM-SIV, ChaCha20-Poly1305, AES-CCM, AES-OCB, and Ascon.
- **Alternative canonicalizations.** Any deterministic serialization may substitute, including JCS (RFC 8785), canonical CBOR, DER, canonical S-expressions, sorted-key TLV encodings, or a fixed-offset binary layout.
- **Alternative bound field sets.** The set of metadata fields placed in the AAD may be enlarged, reduced, or reordered; the binding property holds for whatever fields are included. Explicitly contemplated additional fields include recipient identifiers, jurisdiction or data-residency tags, classification labels, retention schedules, license terms, and audit-endpoint references.
- **Alternative serialization framing.** The metadata may be framed as an array (as above), as an object, as a concatenation of length-prefixed fields, or as a hash of any of these substituted in place of the serialized bytes themselves.
- **Independence from key distribution.** The mechanism is orthogonal to how the CEK reaches recipients, and applies equally where the CEK is wrapped by ECDH plus a KDF plus a key-wrap primitive, by RSA-KEM or other KEM, by a password-based KDF, by a hardware-authenticator-derived key (including WebAuthn/FIDO2 PRF or hmac-secret output), by post-quantum KEMs such as ML-KEM, or where the CEK is split among multiple parties by a threshold or secret-sharing scheme prior to wrapping.
- **Alternative version-gating strategies.** Version selection may be by an explicit version field, by structural detection, by a cryptographic suite identifier, or by an out-of-band profile, provided unknown versions are rejected rather than defaulted.
- **Transport and container variants.** The envelope may be plain JSON, compressed JSON, binary CBOR/MessagePack, or embedded in an existing container format; the mechanism is unchanged.
- **Non-document payloads.** The same construction applies to encrypted messages, streams, database records, backups, firmware images, and archived logs carrying governance metadata.

Worked Example

A minimal envelope encrypted for a single recipient, no quorum, permanent lifetime, unsigned. Bound metadata is `[prev_hash, threshold, lifetime, policy]`; the header hash is computed over the canonical serialization of the complete header.


```
{
  "version": "1.2",
  "doc_id": "01JXXXXXXXXXXXXXXXXXXXXXXX",
  "created_at": 1749420000,
```

```
  "header": {
    "prev_hash": null,
    "threshold": { "required": 1, "of_total": 1 },
    "lifetime": {
      "type": "permanent",
      "expires_at": null,
      "on_expiry": "none",
      "single_use": false,
      "minimum_atl": 1
    },
    "policy": {
      "copy_protection": "standard",
      "watermark_mode": "none",
      "created_on_origin": "https://example.invalid",
      "origin_sig": null
    },
    "access_map": [
      {
        "method": "ecdh-p256",
        "label": "Recipient",
        "fingerprint": "a1b2c3d4e5f6a7b8",
        "share_index": null,
        "wrapped_cek": {
          "eph_spki":  "<base64 — ephemeral public key>",
          "hkdf_salt": "<base64 — 32 random bytes>",
          "ct":        "<base64 — wrapped 256-bit CEK>"
        }
      }
    ],
    "payload_hash": "<base64 — SHA-256 of raw ciphertext bytes>"
  },
  "header_hash":    "<base64 — SHA-256 of canonicalize(header)>",
  "header_sig":     null,
  "header_sig_by":  null,
  "header_sig_key": null,
  "payload": {
    "alg":        "AES-256-GCM",
    "encoding":   "base64+gzip",
    "iv":         "<base64 — 12-byte IV>",
    "ciphertext": "<base64 — AES-256-GCM output over the compressed body>"
  }
}
```

For this envelope the AAD is the UTF-8 encoding of the following single line. Any line breaks below are introduced by page width alone: the canonical form contains **no white-space whatsoever**, and inserting any would change the authenticated bytes.

```
[null,{"of_total":1,"required":1},
{"expires_at":null,"minimum_atl":1,"on_expiry":"none","single_use":false,"type":"permanent"},
```

{"copy_protection":"standard","created_on_origin":"https://example.
invalid","origin_sig":null,"watermark_mode":"none"}]

Note the lexicographic key ordering within each object, produced by canonicalization, and the preserved positional order of the outer array.

Altering `"required": 1` to `"required": 2`, or `"expires_at": null` to a timestamp, changes these bytes and causes decryption of the unmodified ciphertext to fail.

Reference Implementation

A complete, independently runnable reference implementation of the disclosed mechanism is published at:

- Repository: **`github.com/Enthropic-Data-LLC/cd-vanilla-public`** — public, no account required
- Specification: `docs/SPEC.md` (§4.4 AAD binding; §4.1.1 canonical serialization)
- Implementation: `cell-crypto.js` (functions `cellCreate` / `cellOpen`)
- Test suite: `npm test`

The implementation uses only the W3C Web Cryptography API and no third-party cryptographic libraries.

Cellular Defense · Enthropic Data LLC · enthropicdata.com — published as a defensive disclosure. No patent sought or asserted; freely implementable.